This object represents the data shape the consumer expects from a data provider. If the consumer makes a `GET` request to get the details of a user, it only cares if the data of the expected fields are similar to the one we expect; in this case, an integer for the ID and string for the username and full name. If, for some reason, the ID is a string on the provider's side, our contract will fail. We'll cover in more detail how you can use loose matchers with Pact in this chapter.

Even though loose matching is recommended, this recommendation must be considered on a field-to-field basis. If you have specific fields where you care about the data being returned, then don't use loose matchers. Here's another question that the Pact tool recommends you consider: "If I made this looser/tighter, what bugs would I miss/prevent?"

4.1.4 The importance of writing isolated contract tests

From our revised test automation pyramid discussed in chapter 2, contract tests sit above unit tests, so they have similar qualities to a unit test. One of its similarities is that contract tests should be verified in isolation, with no context or dependency from previous interactions. Contract tests that depend on the outcome of previous tests are brittle and will land you in that same state of brittle integration tests.

However, some consumer interactions require providers to be in a certain state. Let's consider an example between a web application and a product API. If we want to write a contract test between a web application sending a `GET` request to retrieve a list of products, this test requires that products exist first. This is great if products always exist on the provider side. However, this might not always be the case. What happens if the provider's data has been refreshed? How can we achieve a reliable contract test?

Contract testing tools such as Pact provide functionalities for setting up provider states to ensure the provider is in the right state for the consumer contract test, as shown in figure 4.2.